

Red Hat Data Grid 8.6

Using the Memcached protocol endpoint with Data Grid

Use the Data Grid Memcached endpoint to interact with remote caches

Last Updated: 2026-06-03

Red Hat Data Grid 8.6 Using the Memcached protocol endpoint with Data Grid

Use the Data Grid Memcached endpoint to interact with remote caches

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

The Memcached protocol endpoint allows clients to connect to one or more Data Grid backed Memcached servers and perform cache operations.

Table of Contents

RED HAT DATA GRID 3

DATA GRID DOCUMENTATION 4

DATA GRID DOWNLOADS 5

MAKING OPEN SOURCE MORE INCLUSIVE 6

CHAPTER 1. MEMCACHED SERVER 7
1.1. CLIENT ENCODING 7
1.2. PROTOCOL DETECTION 7
1.3. SECURITY 7
1.3.1. Text protocol 7
1.3.2. Binary protocol 7
1.3.3. Flush All 7
1.4. UNSUPPORTED FEATURES 8
1.4.1. Individual Statistics 8

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. MEMCACHED SERVER

The Data Grid server distribution contains a server module that implements the Memcached text and binary protocols. This allows Memcached clients to connect with one or several Data Grid backed Memcached servers. These servers can be standalone like Memcached, where each server acts independently and does not communicate with others. They can also be clustered, where the servers replicate or distribute their contents to other Data Grid backed Memcached servers, providing clients with failover capabilities.

The Memcached endpoint is enabled by default on the single-port endpoint. Memcached client connections will automatically be detected and routed to the internal connector.

1.1. CLIENT ENCODING

The Memcached text protocol assumes data values read and written by clients are raw bytes.

It's not possible for a Memcached client to negotiate the data type to obtain data from the server or send data in different formats. The server can optionally be configured to handle values encoded with a certain Media Type. By setting the **client-encoding** attribute in the **memcached-connector** element, the server will return content in this configured format, and clients can also send data in this format.

client-encoding is useful when a single cache is accessed from multiple remote endpoints such as Rest, HotRod, or Memcached. This also allows users to customize the response and request to Memcached text clients. For information about interoperability between endpoints, see the endpoint interoperability documentation.

1.2. PROTOCOL DETECTION

The Data Grid Memcached connector supports both manual and automatic protocol configuration by setting the **protocol** attribute in the **memcached-connector** element to one of **TEXT**, **BINARY** and **AUTO**. Automatic detection depends on whether the Memcached connector is on a dedicated socket binding or is part of a single port endpoint. Detection of text protocol clients in a single port endpoint requires authentication, in order to disambiguate the request from other protocols.

1.3. SECURITY

The Data Grid Memcached connector supports authentication for both the text and binary protocols. Because the two variants implement authentication in a different way, the supported mechanisms depend on the protocol in use.

1.3.1. Text protocol

Authentication in the Memcached text protocol is performed by an initial **set** command where the key is ignored and the value is a concatenation of the username and the password separated by a space. This authentication scheme requires a security realm that supports plain-text.

1.3.2. Binary protocol

Authentication in the Memcached binary protocol is performed via dedicated operations which wrap the SASL challenge/response scheme. This authentication scheme works with all the Data Grid security realm implementations.

1.3.3. Flush All

Even in a clustered environment, the **flush_all** command clears only the Data Grid Memcached server where the call lands. There is no attempt to propagate this flush to other nodes in the cluster. This is done so that the **flush_all** with delay use case can be reproduced with the Data Grid Memcached server. The aim of passing a delay to **flush_all** is so that different Memcached servers can be flushed at different times, and hence avoid overloading the database with requests as a result of all Memcached servers being empty. For more information, see the Memcached text protocol section about **flush_all** .

1.4. UNSUPPORTED FEATURES

This section details features of the Memcached text protocol that are currently not supported by the Data Grid based Memcached implementation.

1.4.1. Individual Statistics

There are differences between the original Memcached implementation, which is C/C++ based, and the Data Grid implementation, which is Java based. There are some general purpose statistics that are not supported. For these unsupported statistics, Data Grid Memcached server always returns **0**.

Unsupported statistics

- pointer_size
- rusage_user
- rusage_system
- bytes
- curr_connections
- total_connections
- connection_structures
- auth_cmds
- auth_errors
- limit_maxbytes
- threads
- conn_yields
- reclaimed

Statistic settings

Statistics setting is not supported.

Settings with Arguments Parameter

Data Grid Memcached server does not support passing any arguments to the **stats** command. If any parameters are passed, the Data Grid Memcached server responds with **CLIENT_ERROR** .

Delete Hold Time Parameter

As Memcached no longer honors the optional hold time parameter for the delete command, the Data Grid based Memcached server does not implement the feature.

Verbosity Command

The verbosity command is not supported because Data Grid logging cannot be simplified to defining the logging level alone.

Meta protocol

The Memcached meta protocol, introduced in Memcached 1.6, is not supported.